

Ejemplos de problemas resueltos con RNA

Ejemplo 1: conjunto de datos diminuto

Considera el siguiente conjunto de datos muy pequeño. La tabla 1 incluye información sobre un puntaje de degustación para cierto queso procesado. Los dos predictores son puntajes para grasa y sal, que indican la presencia relativa de grasa y sal en la muestra de queso en particular (donde 0 es la cantidad mínima posible en el proceso de fabricación y 1 la máxima). La variable de resultado es la preferencia de gusto del consumidor de la muestra de queso, donde gustar (like) o disgustar (dislike) indica si al consumidor le gusta el queso o no.

TABLE 1.		TINY EXAMPLE ON TASTING SCORES FOR SIX CONSUMERS AND TWO PREDICTORS	
Obs.	Fat Score	Salt Score	Acceptance
1	0.2	0.9	like
2	0.1	0.1	dislike
3	0.2	0.4	dislike
4	0.2	0.5	dislike
5	0.4	0.5	like
6	0.3	0.8	like

```
> library(neuralnet)
> df <- read.csv("TinyData.csv")
> df$Like <- df$Acceptance=="like"
> df$Dislike <- df$Acceptance=="dislike"
> set.seed(1)
> nn <- neuralnet(Like + Dislike ~ Salt + Fat, data = df, linear.output = F, hidden = 3)
> # display weights
> nn$weights
[[1]]
[[1]][[1]]
      [,1] [,2] [,3]
[1,] -1.061144 3.057022 3.337952
[2,] 2.326024 -3.408663 -4.293214
[3,] 4.106435 -6.525668 -5.929419

[[1]][[2]]
      [,1] [,2]
[1,] -0.3495333 -1.677856
[2,] 5.8777146 -3.606625
[3,] -5.3529201 5.620330
```

[4,] -6.1115039 6.696287

> # display predictions

> prediction(nn)

Data Error: 0;

\$rep1

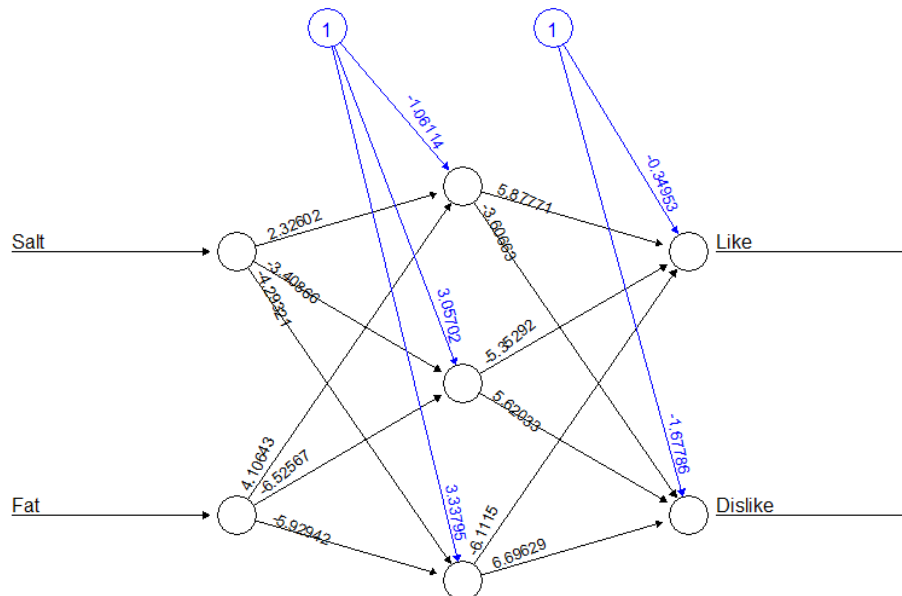
	Salt	Fat	Like	Dislike
1	0.1	0.1	0.0002415536	0.99965512
2	0.4	0.2	0.0344215787	0.96556788
3	0.5	0.2	0.1248666748	0.87816828
4	0.9	0.2	0.9349452648	0.07022732
5	0.8	0.3	0.9591361793	0.04505631
6	0.5	0.4	0.8841904620	0.12672438

\$data

	Salt	Fat	Like	Dislike
1	0.1	0.1	0	1
2	0.4	0.2	0	1
3	0.5	0.2	0	1
4	0.9	0.2	1	0
5	0.8	0.3	1	0
6	0.5	0.4	1	0

> # plot network

> plot(nn, rep="best")



Error: 0.03757 Steps: 76

La matriz de confusión basada en probabilidades, podría decirte que tan bien la red neuronal clasifico.

Ejemplo 2: La función XOR

Intentaremos entrenar una red neuronal en la función booleana XOR. Aquí está la tabla de verdad de la función XOR:

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

Podemos obtener ejemplos para este conjunto de datos muestreando puntos de una distribución normal multivariante centrada en los puntos de coordenadas de cada fila y asociándolos con el valor de verdad de esa fila. Generemos algunos datos:

```
> set.seed(4400)
> library(MASS) # Necesario para muestrear distribuciones gaussianas multivariadas
> library(neuralnet)
> cov <- matrix(c(0.05, 0, 0, 0.05), 2, 2) # Diagonal covariance matrix
> cov
      [,1] [,2]
[1,] 0.05 0.00
[2,] 0.00 0.05

> num.points <- 5000
> first <- mvrnorm(num.points, c(0, 0), cov)
> second <- mvrnorm(num.points, c(0, 1), cov)
> third <- mvrnorm(num.points, c(1, 0), cov)
> fourth <- mvrnorm(num.points, c(1, 1), cov)
> all.points <- rbind(first, second, third, fourth)
> labels <- rep(c(0, 1, 1, 0), each = num.points)
> xor.data <- as.data.frame(cbind(labels, all.points))
> colnames(xor.data) <- c("label", "x", "y")
```

Ahora, veamos algunos de estos:

```
> num.sample.rows <- 10
> display.rows <- sample(1:nrow(xor.data), num.sample.rows)
> xor.data[display.rows, ]
      label      x      y
4762     0 0.0441259 0.09431550
3994     0 -0.0939220 0.19691157
```

```

16283  0 0.7659402 1.05300695
5896   1 0.1799407 0.98880297
18829  0 1.1385206 1.22063978
13805  1 0.9762508 0.14417051
10318  1 1.1859873 -0.18601888
14197  1 0.8909297 -0.50233562
4674   0 -0.5195782 -0.04921216
11070  1 0.9327824 -0.36184650

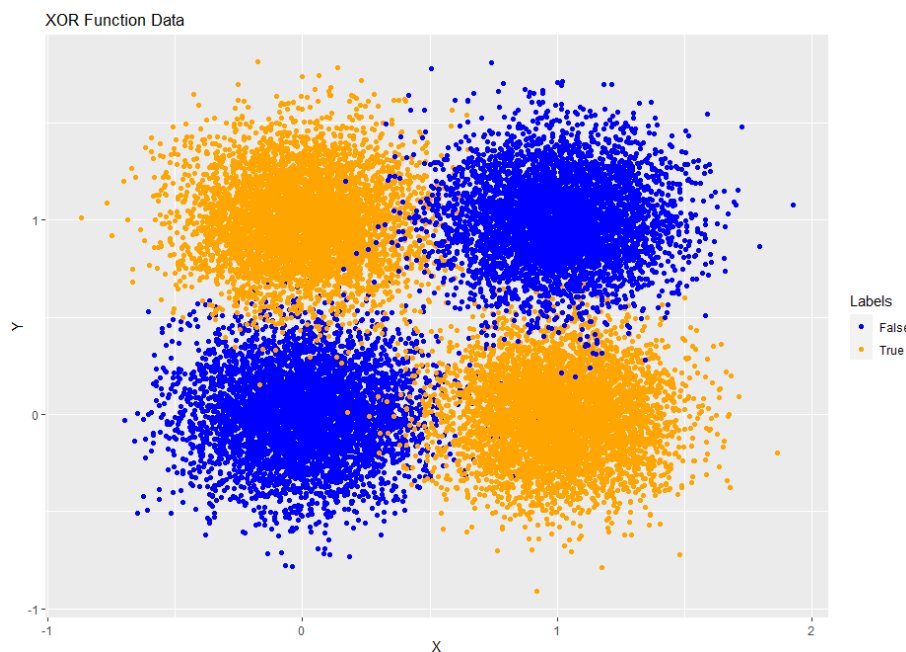
```

Aún mejor, podemos visualizar estos datos:

```

> library(ggplot2)
> ggplot(xor.data, aes(x = x, y = y, color = factor(label))) + geom_point() +
+   scale_color_manual(name = "Labels", values = c("blue", "orange"),
+   labels = c("False", "True")) +
+   ggtitle("XOR Function Data") + xlab("X") + ylab("Y")

```



Ahora estamos listos para ajustar una red neuronal a estos datos utilizando el paquete `neuralnet` (que habíamos cargado anteriormente):

```

> xor.nnet <- neuralnet("label ~ x + y", data = xor.data, threshold = 1,
+   hidden = c(20), # 1 hidden layer with 20 units
+   linear.output = F, # Classification
+   err.fct = "ce", #Error Function
+   act.fct = "logistic") #Activation Function
> cat(sprintf("Best error reached: %f", xor.nnet$result.matrix[c('error'), ]))

```

Best error reached: 1307.592588

```
> test.data <- data.frame(x = c(0, 0, 1, 1),
+   y = c(0, 1, 0, 1),
+   true.label = c(0, 1, 1, 0))
> prediction <- compute(xor.nnet, test.data[, c("x", "y")])$net.result
> cbind(test.data, prediction)
  x y true.label prediction
1 0 0         0 5.938487e-05
2 0 1         1 9.997109e-01
3 1 0         1 9.995874e-01
4 1 1         0 1.212393e-04
>
> more.test.data <- data.frame(x = runif(10), y = runif(10))
> more.predictions <- compute(xor.nnet, more.test.data)$net.result
> cbind(more.test.data, more.predictions)
   x      y more.predictions
1 0.12423632 0.54866919  0.731215159
2 0.90422697 0.80150616  0.002294212
3 0.03837237 0.87045207  0.998904854
4 0.28665360 0.70683817  0.952607583
5 0.73052313 0.70665934  0.033604660
6 0.04074222 0.23769310  0.004275715
7 0.98305029 0.06397681  0.999281403
8 0.76295029 0.88870250  0.004396323
9 0.71210591 0.21761131  0.966707106
10 0.55601637 0.83029019  0.232298505
```

(Una nota es que el parámetro de salida lineal controla si la red neuronal tiene como objetivo hacer regresión o clasificación: **F** implica clasificación y **T** implica regresión.

Además, podríamos haber especificado `rep = 5` porque queríamos hacer el procedimiento de ajuste 5 veces y luego usar los resultados de la mejor iteración. Es posible que debamos hacer esto, ya que los pesos de la red neuronal se inicializan aleatoriamente, lo que significa que una iteración dada puede no conducir a un gran ajuste).

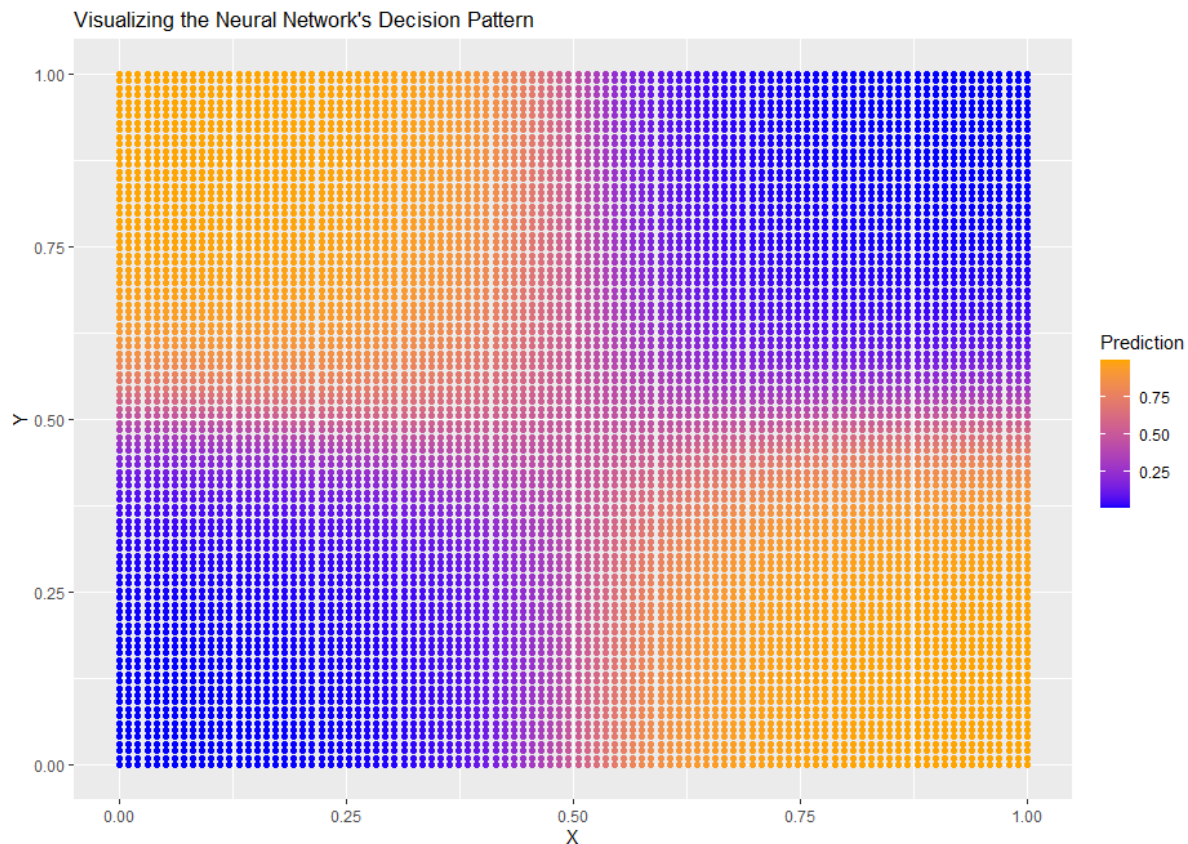
(Otra nota sobre el ajuste de una red neuronal usando el paquete **neuralnet** es que **no da un mensaje útil cuando su ajuste no converge**. El mensaje de error se parece un poco a esto:

```
Error in nrow[w] * ncol[w] : non-numeric argument to binary operator
In addition: Warning message:
In is.na(weights) : is.na() applied to non-(list or vector) of type 'NULL'
```

Esto significa que el valor de los pesos (weights) es NULL, lo que significa que la red no convergió; puedes intentar arreglar esto elevando manualmente el umbral para el cambio en los pesos desde su valor predeterminado de 0.01).

¡Guau, parece que la red neuronal hizo un muy buen trabajo! Podemos visualizar cómo está prediciendo mostrando lo que está prediciendo en una cuadrícula uniforme:

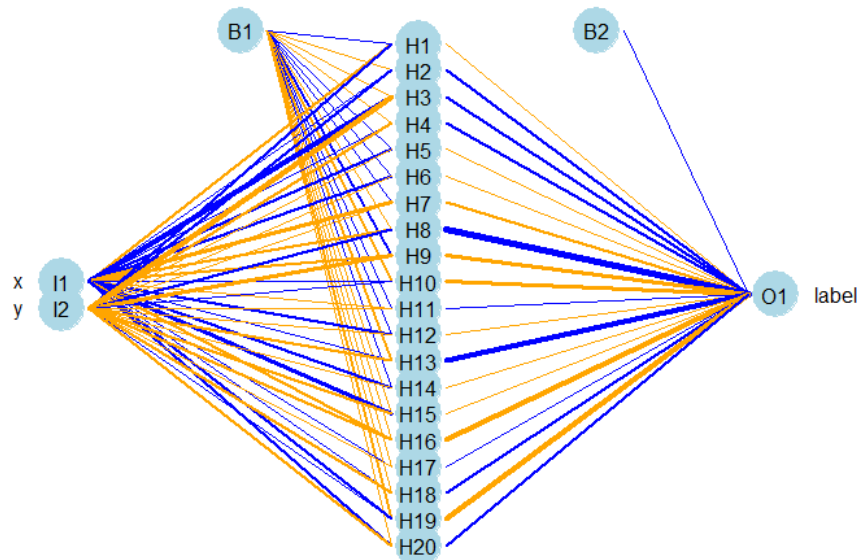
```
> num.interpolating.points <- 100
> x.values <- seq(0, 1, len = num.interpolating.points)
> y.values <- seq(0, 1, len = num.interpolating.points)
> test.points <- as.data.frame(expand.grid(x.values, y.values))
> colnames(test.points) <- c("x", "y")
> predictions <- compute(xor.nnet, test.points)$net.result
> ggplot() + geom_point(aes(x = test.points$x, y = test.points$y, color = predictions)) +
+ scale_color_gradient("Prediction", low = "blue", high = "orange") +
+ ggtitle("Visualizing the Neural Network's Decision Pattern") + xlab("X") + ylab("Y")
```



Una observación importante es que la red neuronal puede encontrar un límite de decisión no lineal a diferencia de su predecesor, el perceptrón, que solo puede encontrar un límite de decisión lineal, como recordarás.

También se puede intentar descubrir qué está haciendo la red neuronal trazando los nodos y viendo los bordes. (Los bordes azules son positivos y los bordes naranjas son negativos; el ancho representa la magnitud).

```
> library(devtools)
Loading required package: usethis
>
source_url('https://gist.githubusercontent.com/fawda123/7471137/raw/466c1474d0a505ff04441270
3516c34f1a4684a5/nnet_plot_update.r')
❗ SHA-1 hash of file is "74c80bd5ddbc17ab3ae5ece9c0ed9beb612e87ef"
> plot.nnet(xor.nnet, pos.col = "blue", neg.col = "orange")
Loading required package: scales
Loading required package: reshape
```



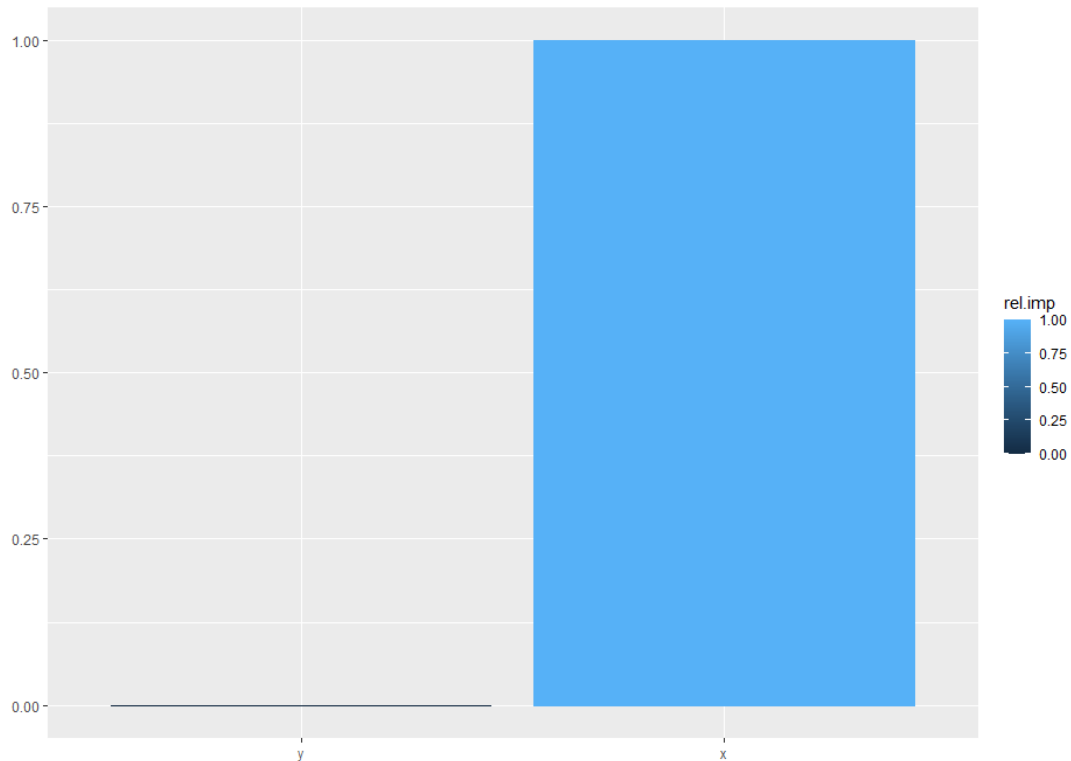
Otro intento de importancia variable se puede ver aquí:

```
> source_gist('6206737')
❗ Sourcing gist "6206737"
❗ SHA-1 hash of file is "7aa33496459a2fe0d3b359351c44fd423928bcfd"
> gar.fun("y", xor.nnet)
Loading required package: plyr
```

Attaching package: 'plyr'

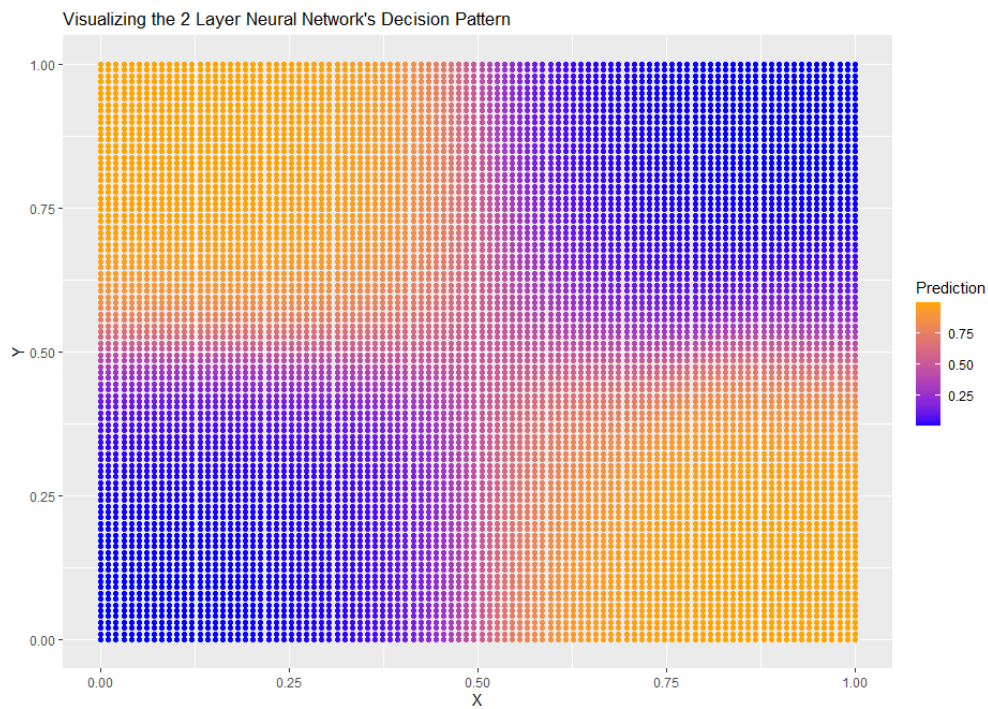
The following objects are masked from 'package:reshape':

rename, round_any

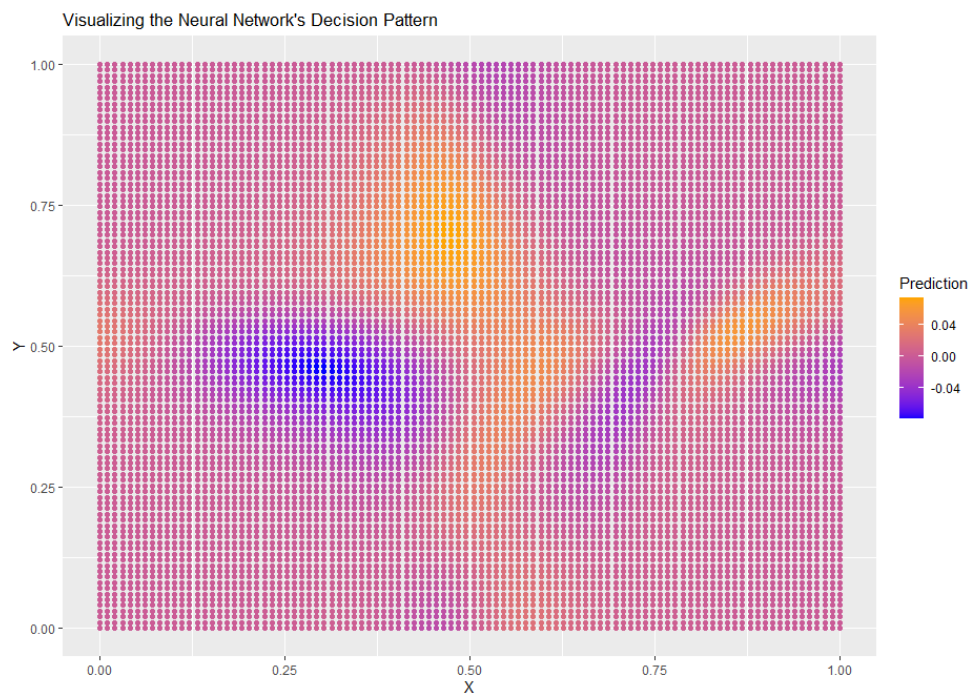


```
> xor.2.nnet <- neuralnet("label ~ x + y", data = xor.data, threshold = 1,
+ hidden = c(20, 20),
+ linear.output = F,
+ err.fct = "ce",
+ act.fct = "logistic") #Activation Function
> cat(sprintf("Best error reached: %f", xor.2.nnet$result.matrix[c('error'), ]))
Best error reached: 1303.165638
```

```
> simple.predictions.2 <- compute(xor.2.nnet, test.data[, c("x", "y")])$net.result
> cbind(test.data, simple.predictions.2)
  x y true.label simple.predictions.2
1 0 0         0      0.0001528870
2 0 1         1      0.9993955915
3 1 0         1      0.9991889222
4 1 1         0      0.0002251395
>
> predictions.2 <- compute(xor.2.nnet, test.points)$net.result
> ggplot() + geom_point(aes(x = test.points$x, y = test.points$y, color = predictions.2)) +
+ scale_color_gradient("Prediction", low = "blue", high = "orange") +
+ ggtitle("Visualizing the 2 Layer Neural Network's Decision Pattern") + xlab("X") + ylab("Y")
```

```
> ggplot() + geom_point(aes(x = test.points$x, y = test.points$y, color = predictions.2 - predictions)) +
+ scale_color_gradient("Prediction", low = "blue", high = "orange") +
+ ggtitle("Visualizing the Neural Network's Decision Pattern") + xlab("X") + ylab("Y")
```



Ejemplo 3: Wisconsin breast cancer data

El último ejemplo se construyó artificialmente, pero ahora probemos las redes neuronales en un conjunto de datos real. El conjunto de datos de cáncer de mama de Wisconsin es un conjunto de datos popular para probar algoritmos de clasificación. Primero, obtengamos los datos:

```
> wbcd.url <- "http://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data"
> wbcd.data <- read.csv(wbcd.url, header = F)
> wbcd.data <- wbcd.data[, -c(1)]
> wbcd.data[, 1] <- as.numeric(wbcd.data[, 1] == "M")
> colnames(wbcd.data)[1] <- "label"
> wbcd.data[1, ]
```

```
label V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17
1 1 17.99 10.38 122.8 1001 0.1184 0.2776 0.3001 0.1471 0.2419 0.07871 1.095 0.9053 8.589 153.4 0.006399
V18 V19 V20 V21 V22 V23 V24 V25 V26 V27 V28 V29 V30 V31 V32
1 0.04904 0.05373 0.01587 0.03003 0.006193 25.38 17.33 184.6 2019 0.1622 0.6656 0.7119 0.2654 0.4601 0.1189
```

La red neuronal ahora se puede entrenar con los datos porque tenemos 1 variable de salida y 30 variables de entrada numéricas. (Las 30 covariables numéricas son la media, el error estándar y el promedio de los 3 valores más grandes de 10 métricas diferentes para el paciente. Dividimos nuestros datos como datos de prueba y entrenamiento.

```
> train.proportion <- 0.8
> train.index <- sample(x = 1:nrow(wbcd.data),
+ size = floor(train.proportion * nrow(wbcd.data)),
+ replace = F)
> wbcd.train.data <- wbcd.data[train.index, ]
> wbcd.test.data <- wbcd.data[-train.index, ]
> wbcd.test.labels <- wbcd.test.data$label
> wbcd.test.data <- subset(wbcd.test.data, select = -c(label))
```

Al igual que antes, ahora podemos calcular redes neuronales; sin embargo, no estamos seguros de cuál es la mejor cantidad de capas (o la cantidad de unidades ocultas en cada capa). Probaremos algunas combinaciones diferentes; una nota es que **tenemos que elevar el umbral de 0.01 porque de lo contrario la red no pensará que ha convergido**. Se encontró experimentalmente que **el valor 2 conduce a resultados decentes**.

```
> formula <- sprintf("%s%s", "label ~ ", paste("V", 3:32, collapse = " + ", sep = ""))
> formula
[1] "label ~ V3 + V4 + V5 + V6 + V7 + V8 + V9 + V10 + V11 + V12 + V13 + V14 + V15 + V16 +
V17 + V18 + V19 + V20 + V21 + V22 + V23 + V24 + V25 + V26 + V27 + V28 + V29 + V30 + V31
+ V32"
```

```

> wbcd.first.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(5), # 1 hidden layer with 5 units
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> wbcd.second.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(10), # 1 hidden layer with 10 units
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> wbcd.third.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(15), # 1 hidden layer with 15 units
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> wbcd.fourth.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(5, 5), # 2 hidden layers with 5 units each
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> wbcd.fifth.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(10, 10), # 2 hidden layers with 10 units each
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> wbcd.sixth.net <- neuralnet(formula, data = wbcd.train.data,
+ hidden = c(15, 15), # 2 hidden layers with 15 units each
+ linear.output = F, rep = 5,
+ err.fct = "ce", act.fct = "logistic", threshold = 2)
> train.scores <- sapply(list(wbcd.first.net, wbcd.second.net, wbcd.third.net,
+ wbcd.fourth.net, wbcd.fifth.net, wbcd.sixth.net),
+ function(x) {min(x$result.matrix[c("error"), ])}))
> cat(paste(c("Training Scores (Logarithmic Loss)\n1 Hidden Layer, 5 Hidden Units:", "1 Hidden
Layer, 10 Hidden Units:", "1
Hidden Layer, 15 Hidden Units:", "2 Hidden Layers, 5 Hidden Units Each:", "2 Hidden Layers, 10
Hidden Units Each:",
+ "2 Hidden Layers, 15 Hidden Units Each:")), train.scores, collapse = "\n"))
Training Scores (Logarithmic Loss)
1 Hidden Layer, 5 Hidden Units: 4.21878878159876
1 Hidden Layer, 10 Hidden Units: 3.42381602226319
1
Hidden Layer, 15 Hidden Units: 0.176731084423234
2 Hidden Layers, 5 Hidden Units Each: 24.1046726285627
2 Hidden Layers, 10 Hidden Units Each: 4.82279867688964
2 Hidden Layers, 15 Hidden Units Each: 6.60993726105256>

```

Ahora, examinemos cómo funcionan las redes neuronales al predecir los datos de prueba:

```

> percentage.correctly.classified <- function(nn, threshold = 0.5) {
+ best <- which.min(nn$result.matrix[c("error"), ])
+ net.predictions <- compute(nn, wbcd.test.data, rep = best)$net.result
+ thresholded.net.predictions <- ifelse(net.predictions > threshold, 1, 0)
+ num.correct <- sum(as.numeric(thresholded.net.predictions == wbcd.test.labels))
+ num.correct / length(wbcd.test.labels)
+ }
> scores <- sapply(list(wbcd.first.net, wbcd.second.net, wbcd.third.net,
+ wbcd.fourth.net, wbcd.fifth.net, wbcd.sixth.net),
+ percentage.correctly.classified)
>
> cat(paste(c("Test Scores (Percentage Correctly Classified)\n1 Hidden Layer, 5 Hidden Units:", "1
Hidden Layer,
+ 10 Hidden Units:", "1 Hidden Layer, 15 Hidden Units:", "2 Hidden Layers, 5 Hidden Units Each:",
"2 Hidden Layers,
+ 10 Hidden Units Each:", "2 Hidden Layers, 15 Hidden Units Each:")), scores, collapse = "\n"))
Test Scores (Percentage Correctly Classified)
1 Hidden Layer, 5 Hidden Units: 0.956140350877193
1 Hidden Layer,
10 Hidden Units: 0.947368421052632
1 Hidden Layer, 15 Hidden Units: 0.964912280701754
2 Hidden Layers, 5 Hidden Units Each: 0.956140350877193
2 Hidden Layers,
10 Hidden Units Each: 0.93859649122807
2 Hidden Layers, 15 Hidden Units Each: 0.885964912280702

```

A partir de estos, podemos ver que el simple hecho de tener más nodos no es útil para obtener predicciones más precisas.